ChatGPT

Mission Planning Suite with Orekit & WorldWind – Requirements Specification (v1.0)

Document Version: 1.0 (September 2025)
Prepared by: *[Project Team]*

Version	Date	Description
1.0	10 Sep 2025	Initial release – Combined end-to-end mission planning suite requirements with Orekit & WorldWind integration.

1. Introduction

This document defines the unified **Software Requirements Specification (SRS)** for an end-to-end **Mission Planning Suite** that integrates advanced flight dynamics and visualization capabilities. The suite is intended to support the entire mission lifecycle – from early design and equipment selection through **Assembly, Integration, and Test (AIT)**, on to operations and mission end-of-life [1]. By merging the original mission planning requirements (equipment selection, standards compliance, AIT support, driver generation, etc.) with new requirements for integrating the **Orekit** and **WorldWind** libraries, this specification provides a comprehensive view of the system's functionality and design.

Orekit (Orbit Extrapolation KIT) is an open-source space dynamics library that provides accurate low-level components for orbital mechanics (e.g. orbits, time, attitude, reference frames) and algorithms for propagation and trajectory analysis [2]. Integrating Orekit enables the suite to perform high-fidelity orbit propagation, maneuver simulation, and related computations as part of mission planning. **NASA WorldWind** is a 3D geospatial visualization SDK that allows rendering of the Earth (or other planets) in interactive 3D within Java applications [3]. By embedding WorldWind, the suite can display satellite orbits, ground tracks, and mission data on a virtual globe, greatly enhancing situational awareness and planning insight.

Overall Goal: Provide mission planners and engineers with a **unified toolset** that covers mission design, hardware selection, compliance checking, and trajectory simulation, all visualized in an interactive 3D environment. This reduces the need to use disparate tools, streamlines the workflow, and ensures consistency across mission phases. The integrated system should facilitate **efficient and streamlined mission planning**, leveraging Orekit for robust flight dynamics calculations and WorldWind for rich visualization [4].

Scope: The Mission Planning Suite is targeted at space missions (e.g. satellite missions, Earth observation projects) and covers both **functional requirements** (capabilities the software must provide) and **non-functional requirements** (quality attributes and constraints). It encompasses planning functions (like selecting spacecraft components and defining mission parameters), analytical functions (simulating orbits, contacts, etc.), compliance features (adhering to standards), and user interface/visualization features. The scope also includes basic architecture guidelines to ensure the system is modular and maintainable, combining formal SRS structure with key design considerations as requested.

Acronyms & Definitions:
- **AIT** – Assembly, Integration, and Test (phase in the mission lifecycle focused on building and testing the spacecraft system).
- **Orekit** – ORbit Extrapolation KIT, a Java library for space mission trajectory and orbit analysis [2].
- **WorldWind** – NASA WorldWind, a software development kit for 3D interactive globe visualization [3].
- **ECSS** – European Cooperation for Space Standardization (space engineering standards) [5].
- **CCSDS** – Consultative Committee for Space Data Systems (international standards organization for space data formats).
- **TLE** – Two-Line Element set, a standard format for orbital parameters of Earth satellites [6].
- **OEM** – Orbit Ephemeris Message, a CCSDS standard file format for orbit trajectory data [6].

2. System Overview and Architecture

System Summary: The Mission Planning Suite is envisioned as a **modular, extensible application** that provides an interactive environment for mission design and analysis. Major functional modules include:

- **Mission Planning Core:** Handles mission configuration, requirements input, and overall scenario management. This core allows users to define mission objectives, select spacecraft configuration, and outline the mission timeline (phases such as launch, operations, decommissioning). It ensures all mission phases (including AIT, launch, operations, end-of-life) are accounted for in planning [1].

- **Equipment/Components Database:** A library of spacecraft components and equipment (e.g. bus platforms, payload instruments, subsystems) with their attributes. Users can browse and select equipment for their mission, and the system will track these selections. This module ensures compatibility of selected components and can enforce **standards compliance** (e.g. only space-qualified or standards-adherent components are chosen).

- **Standards Compliance Module:** Encapsulates rules and checks related to aerospace standards (such as ECSS for European missions or NASA standards for US missions). It provides guidance and verification so that the mission plan and design adhere to required processes and data formats. Compliance with such standards is crucial for ensuring high-quality, compatible, and reliable space systems [7].

- **Flight Dynamics Engine:** Powered by **Orekit**, this module performs all orbit and trajectory calculations. It exposes services for orbit propagation (analytical and numerical), orbital event detection (eclipses, ground station visibility, etc.), maneuver planning, and attitude computations. By using Orekit, the system benefits from tested algorithms and accurate models for time, frames, gravity, drag, etc., enabling high-fidelity simulation of the mission's orbital dynamics [2]. This engine also supports advanced analyses like projecting sensor fields-of-view onto the ground track (for coverage analysis) [8].

- **Visualization & UI Module:** Built around **NASA WorldWind**, this module presents a **3D interactive globe** as well as optional 2D map views for mission visualization [9]. It renders the Earth (or other relevant celestial body) with terrain and map data, and overlays mission-specific visuals: satellite orbits, ground tracks, spacecraft models, ground stations, coverage footprints, etc. Users can interact (zoom, rotate, pan) with the 3D globe and view different layers of information. The UI includes controls for simulation time, scenario configuration, and data display (tables, forms, etc.), enabling a seamless user experience.

- **Integration & API Layer:** To promote modularity, the system provides well-defined **APIs** and extension points. This allows integration with external systems such as mission control software, simulation environments, or custom data sources. (For example, the suite could connect to a flight dynamics server or accept live telemetry from a satellite control system to visualize real-time orbits.) The design echoes approaches like Terma's TRACK system, which offers public APIs to communicate with customer-specific systems [10]. This ensures the suite can be extended or interfaced with other tools without modifying its core.

Architecture: The architecture follows a client-server or layered design within a single application. The **Mission Planning Core** orchestrates interactions between the GUI, the databases, and the computation engines. The **Orekit** library functions may run in the background (possibly in a separate thread or service) to perform calculations on-demand or continuously during simulation. The **WorldWind** component is embedded in the UI for rendering; the Flight Dynamics engine feeds it data (e.g. satellite position updates) which the visualization layer then displays in real time. The **Equipment DB and Standards module** feed into the planning core to provide data and validate decisions at design time. A conceptual diagram of interactions might be:

- **User Interface** ⟷ Mission Planning Core ⟷ (Equipment DB, Standards rules)
- Mission Planning Core ⟷ **Flight Dynamics (Orekit)**
- Mission Planning Core ⟷ **Visualization (WorldWind)**
- Mission Planning Core ⟷ External Systems (via Integration API)

This modular separation ensures that each major capability (e.g. computing orbits vs. displaying maps vs. managing data) can be maintained or upgraded independently. For instance, the WorldWind component could be replaced with another visualization library in future without affecting the Orekit-based computations, due to the clear API boundaries. The use of Java for both Orekit and WorldWind fosters a smooth integration at the code level [11]. Data flow between modules uses standard formats (e.g. orbit ephemeris files, configuration files) so that components remain loosely coupled. The system will maintain a central scenario state (containing spacecraft data, orbit parameters, timeline events, etc.), which is updated by the planning core and observed by the other modules (e.g. the visualization subscribes to updated positions, the standards module checks any new data, etc.).

User Roles: The primary users are **mission planners and systems engineers** who design the mission and need to evaluate various scenarios. They interact through the GUI. Additionally, **AIT engineers** might use the system to ensure the testing campaign is aligned with the plan, and **operators** could use it to generate mission operation plans or visualize orbits. The architecture supports multiple user roles by allowing different configurations or views (for example, a planner might focus on design and analysis features, while an operator might use a subset just for visualization of a finalized plan).

3. Functional Requirements

Below are the detailed functional requirements of the Mission Planning Suite, organized by major capability areas. Each requirement is prefixed with an identifier for traceability (MP for Mission Planning, followed by a section number and item number).

3.1 Mission Design & Equipment Selection

- **MP-DES-1:** *Mission Definition:* The system shall allow users to create a **mission profile** by specifying key parameters such as mission type (e.g. Earth observation, communications, science), target orbits or destinations, mission duration, and objectives. This forms the top-level context that will guide subsequent planning decisions.

- **MP-DES-2:** *Equipment Library:* The system shall provide a **library of spacecraft equipment and components** (for example, satellite bus platforms, payload instruments, communication systems, and ground segment elements). Each entry in the library will include relevant specifications (mass, power, dimensions, interface standards, heritage, etc.). Users must be able to browse and search this library to consider available options for their mission.

- **MP-DES-3:** *Equipment Selection:* Users shall be able to **select and configure mission components** from the library to build their mission architecture. For instance, a user can choose a particular satellite bus model and then add instruments or subsystems to it. The system should support multiple alternatives (trade studies), allowing users to compare different equipment selections side-by-side (e.g. comparing two different payloads or two bus options).

- **MP-DES-4:** *Compatibility Checking:* The system shall automatically check for **compatibility and constraints** when equipment is selected. This includes verifying that components can fit together and meet mission constraints (e.g. total mass not exceeding launch vehicle capacity, power consumption within available power budget, data interfaces matching). If a selection violates a constraint (for example, a payload requiring more power than the bus can provide), the system will flag it and suggest modifications if possible.

- **MP-DES-5:** *Mission Phases Planning:* The mission design functionality shall cover all relevant **mission phases**, including early deployment, nominal operations, and decommissioning. The user should be able to outline phase-specific activities or configurations (for example, an instrument might be off during launch phase, or a deployable antenna extends in early orbit phase). All mission phases, **including AIT, pre-launch, launch, operations, and end-of-life, should be supported in the planning timeline** [1], ensuring nothing is overlooked in the design.

- **MP-DES-6:** *Traceability:* The system shall maintain traceability between mission requirements and selected design elements. For each piece of equipment or configuration choice, users should be able to link it to the driving requirement or mission objective. This creates a clear rationale for design decisions and helps during reviews (e.g. design reviews or requirements verification).

- **MP-DES-7:** *Versioning of Design:* The mission design module shall allow versioning of the mission configuration. Users can save different baseline versions or drafts of the mission plan as it evolves (for example, a Version 1.0 for Phase A, updated Version 2.0 for Phase B, etc., reflecting the project lifecycle). Each version should record the date and author of changes, supporting a controlled evolution of the design baseline.

3.2 Standards Compliance & Documentation

- **MP-STD-1:** *Standards Repository:* The system shall include or connect to a repository of relevant **space industry standards and guidelines** that impact mission planning. This may cover standards for data formatting (e.g. CCSDS telemetry/telecommand standards), environmental testing standards, safety and quality standards (ECSS or NASA directives), and any domain-specific standards (e.g. Earth observation data formats).

- **MP-STD-2:** *Compliance Enforcement:* The system shall assist users in **complying with standards** by automatically checking the mission plan and design against known requirements from these standards. For example, if ECSS standards require certain documentation or analysis at specific project phases, the system should prompt the user or provide templates. If a selected

component is not compliant (e.g. a non-space-rated component for a space mission), the system should warn the user.

- **MP-STD-3:** *Standards Guidance:* For each standard that is relevant, the system should provide brief guidance or references. For instance, if a mission is an Earth observation mission under an ESA contract, the suite should highlight that **ECSS standards must be followed**, and list the applicable ECSS documents (engineering, product assurance, etc.). This helps educate users on what is required. Being **compliant with ECSS or similar standards ensures high-quality and compatibility of the mission design** [7], so the system will encourage and facilitate that compliance.

- **MP-STD-4:** *Document Generation:* The system shall be capable of generating standard **documentation outputs** to support mission reviews and verifications. This includes documents like Requirements Compliance Matrix, Interface Control Documents, Test Plans, etc., populated with data from the mission plan. The format of these documents should align with industry standards or provided templates. For example, generating a mission plan document that can be used for a Preliminary Design Review (PDR) or a Mission Requirements Document, ensuring all required sections are present.

- **MP-STD-5:** *Data Format Standards:* The system shall import and export mission data in standard formats where applicable. For orbital data, it should support formats such as **Two-Line Element (TLE) sets and CCSDS Orbit Ephemeris Messages (OEM) for trajectory data** [6]. For mission timelines or command sequences, it could support formats like CCSDS Mission Operations services or other exchange schemas. This ensures interoperability with external tools and compliance with standards for data exchange.

- **MP-STD-6:** *Audit Trail:* In support of quality standards, the system shall maintain an **audit log** of changes made to the mission plan (who changed what and when). This is important for configuration control and for demonstrating compliance to processes (e.g. configuration management standards). It also ties into versioning, as described in MP-DES-7.

3.3 Assembly, Integration & Test (AIT) Support

- **MP-AIT-1:** *AIT Planning:* The system shall include features to plan and track the **Assembly, Integration, and Testing (AIT)** activities for the mission's hardware and software. Users should be able to define the sequence of integration steps and tests that the spacecraft (and ground segment, if applicable) will undergo. For example, define stages such as unit-level testing, subsystem integration, environmental tests (vibration, thermal vacuum), up to the final system tests and launch readiness.

- **MP-AIT-2:** *Link to Design:* The AIT plan should be directly linked to the mission design data. Each piece of equipment or subsystem selected in the design (from section 3.1) should have corresponding **AIT procedures** (e.g. if a certain payload is part of the spacecraft, the plan should include testing that payload). The system should either auto-generate placeholder test activities based on components (using standard test flows for that type of component) or provide templates to the user.

- **MP-AIT-3:** *Standards in AIT:* The system shall ensure AIT activities comply with relevant standards and best practices (for example, ECSS standards for testing, or NASA STD for test procedures). It will include standard test requirements such as doing functional tests before and after

environmental tests, performing rehearsals, etc. **All mission phases shall be supported, including the AIT phase** as an integral part of the mission lifecycle [1].

- **MP-AIT-4:** *Scheduling & Resources:* The system shall allow AIT activities to be scheduled on a timeline (with dates or relative times before launch) and assign resources (facilities, staff, test equipment). This effectively creates an **AIT schedule** that can be integrated with the overall project schedule. It should be possible to identify critical path items or potential schedule risks in the AIT flow.

- **MP-AIT-5:** *Driver/Script Generation for Tests:* Where feasible, the system shall assist in generating test **drivers or scripts** for the AIT phase. For instance, if the suite knows the interfaces of a component (from the equipment library) and the test to be performed, it could generate a basic test script or software driver to command that component during a test (e.g. send commands to turn on a payload and collect telemetry). This automated **driver generation** reduces manual effort and ensures consistency between the planning and execution of tests.

- **MP-AIT-6:** *Results Capture:* The system should allow AIT results (test outcomes, measured performance) to be recorded and linked back to the plan. Although executing tests is outside the planning tool's direct scope, having placeholders to input results or to verify that "Test X passed" vs requirement can help ensure the plan is updated with actual performance. This closes the loop between planning and real-world verification.

3.4 Software Driver Generation & Automation

- **MP-DRV-1:** *Interface Definition:* For each piece of hardware or subsystem in the mission design, the system shall capture its **interface requirements** (telemetry/telecommand definitions, electrical interfaces, protocols). This information is essential for generating software drivers or stubs.

- **MP-DRV-2:** *Automatic Driver Generation:* The system shall be capable of generating **software drivers or stub code** that can interact with the mission's hardware components or simulations thereof. For example, if the mission includes a specific sensor unit that communicates via a known protocol, the suite can generate a basic driver in a target language (such as C/C++ or a scripting language) to interface with that sensor. The purpose is to streamline the development of mission-specific test software or even flight software by providing a starting point that is consistent with the planned design.

- **MP-DRV-3:** *Simulation Drivers:* In cases where physical hardware is not yet available, the system should help generate **simulation drivers**. This might include creating simulated data streams or models for how a device would behave. For instance, generating a software module that simulates the sensor outputs based on the mission scenario (perhaps using data from Orekit simulations to simulate readings like Earth observation instrument swath data, etc.).

- **MP-DRV-4:** *Customization:* The generated drivers should be intended as a baseline – the system shall allow developers/engineers to customize and extend the auto-generated code. Clear documentation should accompany the generated drivers (comments explaining each function, etc.) to facilitate further development.

- **MP-DRV-5:** *Consistency with Plan:* Any assumptions or parameters used in driver generation (such as device IDs, data rates, etc.) shall be drawn from the mission plan to ensure consistency. If the

mission plan is updated (for example, a component is replaced with a different model that has a different interface), the system should flag that previously generated drivers may need to be regenerated or updated.

3.5 Flight Dynamics & Trajectory Analysis (Orekit Integration)

- **MP-FD-1:** *Orbit Propagation:* The system shall integrate the **Orekit** library to perform orbit propagation and trajectory analysis. Users can define initial orbit parameters for spacecraft (e.g. via classical orbital elements, state vectors, or import from TLE) and the system will compute the spacecraft's position and velocity over time. Both **analytical propagation (e.g. Keplerian, SGP4 for TLE) and numerical integration** with perturbation models should be available [12] [13], to cater to different fidelity needs.

- **MP-FD-2:** *Trajectory Events:* The flight dynamics engine shall detect and provide notifications for key **orbital events**. This includes ground station visibility (access periods), eclipse entry/exit (penumbra/umbra events), ascending/descending node crossings, and possibly collision conjunction warnings (if provided tracking data of other objects). For example, given ground station locations, the system can determine Acquisition of Signal (AOS) and Loss of Signal (LOS) times for each pass [14]. These events should be available for visualization and for driving mission planning decisions (e.g. scheduling communications or payload operations only during visibility windows).

- **MP-FD-3:** *Constellation Support:* The system shall support analysis of **multiple spacecraft (constellations)** simultaneously. It should be possible to propagate multiple orbits in parallel and calculate relative configurations (e.g. inter-satellite distances, cross-link visibility). The planning suite should handle constellation-specific setups like common orbital planes or phased deployment. Visualization shall reflect all satellites in the constellation (see section 3.6) [15].

- **MP-FD-4:** *Maneuver Planning:* The system shall allow users to plan orbital **maneuvers** (such as delta-v burns) and see their effect on trajectories. Orekit's capabilities for impulsive and continuous maneuvers should be exposed [16]. For instance, a user can specify a propulsion maneuver at a certain time (or orbit position) and the system will propagate the post-maneuver orbit. This is useful for mission plan scenarios like orbit insertion, phasing or station-keeping maneuvers, and de-orbit burns.

- **MP-FD-5:** *Attitude and Pointing:* The flight dynamics module shall support basic **attitude simulation** for spacecraft, especially to evaluate pointing requirements. Users might define an attitude profile (sun-pointing, nadir-pointing, inertial hold, etc.), and the system will provide orientation information. This is important for payload operations planning (e.g. a camera instrument that must point to targets). If detailed attitude dynamics are needed (e.g. attitude control maneuvers), the system can use Orekit's attitude APIs to model them [2].

- **MP-FD-6:** *Field-of-View & Coverage Analysis:* **Sensor coverage analysis** shall be supported. Users can input sensor Field-of-View (FoV) geometry (e.g. cone angle, shape) for instruments or antennas on the spacecraft. The flight dynamics engine (Orekit) will compute the projection of these FoVs onto Earth's surface and determine coverage areas or ground footprints [8]. For an Earth observation mission, for example, the system can show the ground **swath path** of an imaging instrument over time [17]. It can also calculate when targets or Areas of Interest (AOIs) fall within the sensor FoV given the orbit, aiding in planning observation schedules.

- **MP-FD-7:** *Accuracy and Timing:* The system's computations shall take into account high-precision time systems and reference frames as provided by Orekit (e.g. handling of UTC leap seconds, conversion between terrestrial and inertial frames). This ensures that the mission analysis is accurate and aligned with real-world physics and timing. The use of Orekit's proven algorithms means the results (such as orbit ephemerides, event timings) are **accurate and reliable** for mission planning needs [2].

- **MP-FD-8:** *User Interaction with Trajectories:* The user interface shall allow planners to easily set up and modify orbital parameters. This might include a form to enter classical orbital elements, or a tool to import TLE files. The user can also drag a timeline slider to different epochs and see the computed state of the satellite at that time. The system should update visualizations and data readouts (like latitude/longitude, altitude, velocity, etc.) as the time is adjusted. In essence, the flight dynamics computations should be responsive to user input to facilitate interactive mission analysis.

3.6 Visualization & User Interface (WorldWind Integration)

- **MP-VIS-1:** *3D Globe and 2D Map:* The system shall provide an **interactive 3D Earth globe** view for visualization, using WorldWind or an equivalent engine [9]. Users can zoom, rotate, and tilt the globe to view the spacecraft orbits from any angle. Additionally, a 2D map projection view (flat Earth map) shall be available for certain tasks like coverage maps or ground track plots [9]. The UI should allow switching between 3D and 2D modes seamlessly.

- **MP-VIS-2:** *Orbit and Ground Track Visualization:* The orbits of spacecraft shall be drawn on the globe in real time. This includes rendering the **ground tracks** (the path over Earth's surface directly below the satellite) as well as the orbit arcs in space around the Earth [15]. As the simulation time progresses (or is stepped through), the satellite icon moves along the orbit and the ground track updates accordingly. The system should also display the subsatellite point (current ground position) continuously.

- **MP-VIS-3:** *Multiple Objects:* The visualization shall support **multiple satellites and objects** concurrently. In a constellation scenario, all satellites are shown with distinct markers or models, and their individual orbits/ground tracks can be color-coded. The system should handle visualization of a large number of objects without clutter; for example, allowing toggling visibility of certain groups or focusing on one spacecraft at a time. Constellation visualization support is important for missions with many satellites [15].

- **MP-VIS-4:** *3D Models:* It shall be possible to attach 3D models to the spacecraft for a more realistic view. For example, a satellite 3D model (if available) can be shown instead of a generic icon, oriented according to the spacecraft attitude. Likewise, ground stations might have icons or models on the globe. The system should allow loading of common model formats (or provide a library of generic models). This is mentioned as **3D model support for satellites and ground stations** in similar tools [15].

- **MP-VIS-5:** *Ground Stations & Links:* The system shall allow the user to input ground station locations (or use a predefined list). These stations will be visualized on the Earth map. The tool will display the **visibility footprint** of each ground station (the area on Earth or the space volume from which a satellite is visible, typically a cone defined by elevation mask) [14]. When a satellite is within line-of-sight of a ground station, the visualization can optionally draw a line or highlight the connection. It should also visually indicate AOS/LOS events, e.g. flashing or color change of the station or satellite marker when contact begins/ends.

- **MP-VIS-6:** *Field of View Cones:* For onboard sensors with defined FoVs (from MP-FD-6), the visualization shall depict the **instrument coverage**. This could be a cone emanating from the satellite model showing the area of Earth it covers, and/or directly drawing the ground footprint shape of the FoV [17]. As the satellite moves, this footprint sweeps out a swath on Earth's surface, which should be rendered for the user to see where and when the satellite will observe or communicate. This feature is particularly useful for Earth observation missions to see which ground areas will be imaged.

- **MP-VIS-7:** *Areas of Interest (AOI):* Users shall be able to specify target regions or points of interest on the Earth (or other body) – for example, a region to image or a ground site to communicate with. The system will mark these **Areas of Interest (AOIs)** on the map [18]. Coupled with the orbital tracks and sensor footprints, the UI can visually indicate when a satellite's coverage overlaps an AOI (meaning an opportunity to collect data or communicate). AOIs can be defined by coordinates or by selecting on the map.

- **MP-VIS-8:** *Event Timeline & Animation:* The UI shall include a timeline control that allows the user to play, pause, or scrub through the mission timeline. As time progresses, all visual elements update (satellite positions, ground contacts, etc.). The timeline can be annotated with events (e.g. "Deploy solar panels" at T+1 hour, or "Begin data collection" during a pass). The user can jump to any event or time. The simulation speed can be adjustable (real-time, faster, or step-by-step). This helps in analyzing the mission dynamics over time, not just in static snapshots.

- **MP-VIS-9:** *User Interaction & Info Display:* The visualization should support interactive picking – for example, clicking on a satellite or ground station should bring up a small info window (or "card") with details about that object (name, current orbital parameters, status) [9]. Similarly, if multiple objects are near the cursor, the user can select which one. There should be the ability to toggle various layers (e.g., turn on/off orbit traces, ground station visibility, sensor cones, etc.) to reduce clutter.

- **MP-VIS-10:** *Global Data and Terrain:* WorldWind by default provides the capability to show Earth imagery and terrain. The system shall make use of this to give context (for example, seeing satellite ground tracks over actual maps). If the mission involves other celestial bodies (Moon, Mars), the visualization should be able to switch the globe to those worlds, leveraging WorldWind's support for other planetary globes if needed [19]. High-level global data (coastlines, political boundaries, etc.) should be included for reference.

- **MP-VIS-11:** *Performance and Responsiveness:* The visualization must remain responsive even as complex data is displayed. Using WorldWind's built-in optimizations (level-of-detail imagery, caching) ensures smooth rendering of large data sets [20]. The system should be tested with scenarios such as 100 satellites to verify that orbit drawing and updates remain fluid. If necessary, the visualization will simplify or limit detail (e.g., not drawing the entire orbit trail for all satellites at once) to maintain performance. This requirement ties into non-functional performance criteria (see section 4).

- **MP-VIS-12:** *Customization & Extensibility:* The visualization module shall be designed to allow adding custom layers. For example, a mission might want to visualize additional data like radiation zones (the Van Allen belts), or launch ground tracks, etc. The system should permit plugin of new visualization layers by developers (leveraging WorldWind's Layer architecture [21]). Also, the color and style of existing elements (orbit lines, icons) should be configurable by the user to suit different preferences or presentation needs.

3.7 External Interfaces & Data Management

- **MP-INT-1:** *Orbit Data Import/Export:* The system shall be able to **import external orbit data** for use in planning. This includes reading TLE files or other ephemeris files provided by users (e.g., an externally generated trajectory) – these will be converted into the internal scenario. The suite should also export trajectory data (for example, after planning a maneuver, export the updated ephemeris as a CCSDS OEM file for others to use) [6].

- **MP-INT-2:** *Integration with Mission Control Systems:* The suite should interface with mission operations software for the execution phase. For instance, after planning, a **command sequence** or timeline could be exported to a Mission Control System (MCS) in a format like CCSDS Schedule files or compatible scripts. Conversely, the tool might import operational constraints from the MCS (like station schedules or flight rules) to ensure the plan is realistic. While full integration may be project-specific, the suite provides **APIs to enable communication with external systems** (similar to how TRACK can connect to a control system like CCS5) [10].

- **MP-INT-3:** *Simulator Integration:* The system shall allow coupling with simulation frameworks (such as SIMSAT or custom mission simulators). This means the mission planning data (scenario configuration, events) can be sent to a simulator to perform detailed analysis, and results (telemetry profiles, etc.) can be fed back for assessment. The design with a clear integration layer (section 2) ensures that hooking up such simulators via APIs or data exchange is feasible without core changes.

- **MP-INT-4:** *Database and Persistence:* All mission planning data (design selections, parameters, orbits, timelines, etc.) shall be stored in a persistent **project file or database**. Users can save a project and reopen it later, with all information intact. The system might use an underlying relational database or structured file (like XML/JSON) for storing this data. It should support **multiple projects** so users can manage several mission scenarios separately.

- **MP-INT-5:** *Collaboration:* If multiple team members need to work on the plan, the system should support a mode of collaboration. For example, it could allow multi-user access to the project database or provide a merge mechanism for changes (this could be a future/advanced feature). At minimum, the system should lock or warn if a project file is being edited by someone else concurrently, to avoid version conflicts.

- **MP-INT-6:** *Security and Access:* The system shall implement basic security for its data, especially if it's a multi-user or networked application. Users may have roles (planner, administrator, viewer) with different permissions. Mission data should be protected from unauthorized access or editing. If the application is deployed in an environment with internet connectivity (for updates or pulling data like TLEs from Celestrak, etc.), it should use secure protocols.

- **MP-INT-7:** *Logging and Error Handling:* The system shall log significant actions and errors. This includes logging any issues in integration (like failure to fetch data from an external system, or simulation errors from Orekit). Logs help troubleshoot problems and are often required for quality processes.

- **MP-INT-8:** *Extensibility:* The suite's architecture will allow new modules or plugins to be added (as discussed earlier). This means new external interfaces can be integrated without modifying the core. For example, if a new standard for orbit data comes out, a plugin could be created to

import/export that format, using the public API. This future-proofs the system against evolving needs.

4. Non-Functional Requirements

Beyond the specific capabilities, the system must meet various quality attributes and constraints:

- **Performance:** The application should handle computational tasks and visualization updates efficiently. **Orbit propagation calculations must be optimized** to provide near real-time feedback to the user (leveraging Orekit's efficient algorithms and propagation integrators). The UI should remain responsive, with frame update rates sufficient for smooth visualization (targeting e.g. 30 frames per second for animation). Even with a full constellation scenario (say dozens of satellites and multiple ground stations), the performance should be acceptable on modern hardware (any heavy computations can be done in background threads).

- **Accuracy & Precision:** Given the mission-critical nature of planning, the numerical computations (orbits, event timings) must be very accurate. Orekit ensures a high level of accuracy in flight dynamics [2], but the system must also use appropriate data (e.g. up-to-date Earth orientation parameters, atmospheric models if needed) to avoid errors. Any simplifications (like using two-body propagation) should be clearly indicated to the user. The goal is that results from the planning tool can be trusted for real mission decisions (with appropriate margins).

- **Reliability & Robustness:** The software should be thoroughly tested to avoid crashes or corrupt data. It must handle error conditions gracefully (for example, if a user inputs an invalid orbit or a component selection that's not possible, the system should flag it without crashing). Autosave or backup features are desirable to prevent data loss. The system should also be stable in long runs (one might keep it running during a design session for hours).

- **Usability:** The target users are engineers and mission planners who may not be software experts, so the UI must be intuitive and well-organized. Following UI design best practices for engineering tools, the interface should minimize clutter but allow quick access to needed functions. Clear visualization (with legends, labels, and tooltips) is important for user understanding. User feedback (like highlighting a selected satellite's orbit, or showing a loading indicator during a long computation) will improve usability. Short, context-sensitive help or tutorials for complex features (like how to set up a maneuver simulation) can be included.

- **Compatibility & Portability:** The system is built on Java and should be **cross-platform**. It shall run on commonly used OS such as Windows and Linux without requiring platform-specific modifications [22]. This broad compatibility is important as different organizations use different environments. The software should also not require extremely specialized hardware; a standard engineering workstation (with decent CPU/GPU) should suffice. If using WorldWind Java, a graphics card supporting OpenGL is needed, which is typical of most modern machines.

- **Modularity & Maintainability:** By design, the system is modular (as described in section 2). This modularity must be maintained in implementation to ensure that updates or bug fixes in one part (e.g. visualization) do not unintentionally affect another (e.g. computations). The code should be documented and structured, following good software engineering practices. Ideally, each requirement in this document can be mapped to a module or class responsibility, aiding verification and future maintenance.

- **Extensibility:** The architecture should allow the incorporation of new features with minimal rework. For example, if a future requirement is to integrate a different visualization engine or to support human spaceflight mission planning, the existing structure should accommodate this by adding new modules or replacing components. The use of standardized interfaces and APIs (for instance, the integration layer) is key to this flexibility [10].

- **Standards Compliance (Quality):** Not only should the mission plan comply with external standards (as per functional reqs), but the software development itself could adhere to relevant standards (e.g. DO-178C if it were flight software, or more relevantly ECSS-E-ST-40C for software engineering in space systems). While detailing this is beyond scope, it implies things like having a requirements verification matrix (each requirement tested), using configuration control for the code, etc., to deliver a high-quality product.

- **Security:** If the tool is used in environments with sensitive mission data, it should ensure data confidentiality (e.g. by storing project files in secure format if needed, or respecting IT security guidelines). If a networked version exists, communications should be encrypted. Access control (user login with appropriate privileges) might be required for multi-user installations.

- **Scalability:** The design should consider scalability in terms of mission complexity. Perhaps initially a single mission scenario is handled in memory, but future needs might involve multi-mission or campaign planning. The data model and performance should scale to handle more objects, longer timelines, or higher fidelity models by possibly toggling level of detail or using more computing resources (parallel processing for multiple satellites propagation, etc.).

- **Internationalization:** (Optional) If the tool will be used internationally, it might need support for multiple languages for the UI or different units (e.g. metric vs imperial in some cases). At minimum, the software should use consistent units (SI units for scientific calculation, which is standard in space) and clearly display them.

- **Support & Maintainability:** It is expected that this software will be used over multiple mission cycles, potentially over years. Therefore, it should be easy to update (modular updates, well-documented code for new developers to pick up). Logging and error reporting should facilitate quick diagnosis of any issues that users encounter in the field.

- **Documentation & Training:** The project shall deliver user documentation (user manual, tutorials) and possibly an on-boarding training program for new users. This is to ensure that the rich feature set (covering everything from design to visualization) is fully utilized by the teams adopting the tool.

In summary, the non-functional requirements aim to ensure the Mission Planning Suite is **efficient, accurate, user-friendly, and robust**, aligning with the professional standards expected in aerospace software. The integration of well-established libraries like Orekit and WorldWind not only adds capability but also contributes to these quality goals, since they come with their own optimizations and reliability (Orekit is a widely used library in flight dynamics [4], and WorldWind provides a solid visualization core). The development team will need to verify each of these non-functional aspects through testing and quality assurance processes as the software is built.

5. System Evolution and Design Considerations

(Informative section – not formal requirements.)

This unified specification has combined the initial mission planning requirements with new integration capabilities. Going forward, careful system design is required to meet all the above requirements. Key design considerations include:

- Using a **modular architecture** where Orekit and WorldWind components are loosely coupled with the main application logic, enabling independent upgrades. For instance, if a new version of Orekit provides improved algorithms, it can be integrated without altering the visualization module, as long as interfaces remain consistent.

- Ensuring **validation and verification** of each requirement: for example, test cases will be prepared to validate orbit propagation accuracy against known references, or to ensure an equipment selection that violates a constraint indeed triggers a warning.

- Planning for **future expansion**: The space industry evolves (e.g., new standards like the upcoming revisions, new mission profiles like megaconstellations or lunar missions). The suite should be built with extension in mind, such as plugin support or configurable rules. As noted, providing public APIs and modular design will help keep the tool relevant for new use-cases [10].

- Balancing detail and usability: The requirements listed are extensive, but it is important that the user experience remains coherent. The design should avoid overwhelming the user. One approach is a **tiered interface**: basic mode for essential planning tasks, and advanced panels for detailed analysis (like fine-tuning maneuver parameters or viewing raw data).

Finally, while this document serves as a comprehensive SRS, the implementation team should also consult domain experts and end-users to refine these requirements and prioritize features for iterative development. By meeting the above requirements, the Mission Planning Suite will provide a powerful integrated environment for mission designers – combining rigorous engineering analysis with intuitive visualization – ultimately increasing efficiency and confidence in the mission planning process.

---

[1] ECSS-E-ST-50C Rev.2
https://ecss.nl/wp-content/uploads/2024/12/ECSS-E-ST-50C-Rev.2(5December2024).pdf

[2] [8] [12] [13] [16] Overview – Orekit
https://www.orekit.org/site-orekit-development/

[3] [19] [20] [21] Overview (NASA WorldWind)
https://worldwind.arc.nasa.gov/autodocs/WorldWindJava/

[4] Mission planning and analyses for phase C and D of an earth observation mission
https://www.politesi.polimi.it/handle/10589/209875

[5] [7] ECSS Standards and the ECLIPSE Software Suite - ECLIPSE Suite
https://www.eclipsesuite.com/ecss-standards-and-the-eclipse-software-suite/

[6] [9] [10] [11] [14] [15] [17] [18] [22] terma.com
https://www.terma.com/media/j15e2zlw/data-sheet-track.pdf